

Teach Module 8: Java Project Structure and Modularization

Source used only for topic identification: docs/Java Software Engineer bootcamp.docx

Teaching note: This lesson does not reuse the course material for teaching. It uses the course outline only to identify that Module 8 covers Java project structure, package layout, module boundaries, dependency separation, organization patterns, collaboration standards, and a lab.

Module 8 Overview

When Java projects get bigger, the problem is no longer just: "Can I write a class?"

The problem becomes:

How do I organize many classes so the project stays understandable, testable, and safe to change?

That is what project structure and modularization solve.

By the end of this module, learners should be able to:

- Recognize a standard Java project layout.
- Place source code, resources, and tests in the right folders.
- Use packages to organize related classes.
- Compare layer-based and feature-based organization.
- Split large responsibilities into focused classes.
- Separate dependencies using interfaces.
- Follow collaboration standards that make a project easier for a team to maintain.

1. Standard Java Project Layout

Most professional Java projects use a Maven-style layout, even if they are built with Gradle.

A typical project looks like this:

```
banking-app/  
  pom.xml  
  src/  
    main/  
      java/  
        com/example/banking/  
          Main.java  
      resources/  
        application.properties  
    test/  
      java/  
        com/example/banking/  
          AccountServiceTest.java  
      resources/  
        test-data.json
```

Important folders:

```
src/main/java
```

Production Java code goes here.

```
src/main/resources
```

Configuration and non-Java files go here, such as `.properties` , `.yml` , SQL files, templates, or static resources.

```
src/test/java
```

Test code goes here.

```
src/test/resources
```

Files used only by tests go here.

The simple rule:

Production code and test code should be clearly separated.

2. Packages

A package is Java's way of grouping related classes.

Example:

```
package com.example.banking.accounts;

public class Account {
}
```

The folder should match the package:

```
src/main/java/com/example/banking/accounts/Account.java
```

Packages help with:

- avoiding name conflicts
- grouping related code
- controlling visibility
- making the project easier to navigate

A common package root uses reverse domain naming:

```
com.company.project
```

Examples:

```
com.bank.loanprocessing
com.acme.inventory
com.example.orders
```

3. Organizing by Feature vs Organizing by Technical Layer

There are two common ways to organize Java code.

Option A: By Technical Layer

```
com/example/banking/  
  controller/  
    AccountController.java  
  service/  
    AccountService.java  
  repository/  
    AccountRepository.java  
  model/  
    Account.java
```

This groups classes by what kind of thing they are.

This is common in Spring applications, but in Module 8 the focus is only the basic architecture idea.

Option B: By Feature

```
com/example/banking/  
  accounts/  
    Account.java  
    AccountService.java  
    AccountRepository.java  
  transfers/  
    Transfer.java  
    TransferService.java  
    TransferRepository.java
```

This groups classes by business area.

For larger systems, feature-based organization often scales better because all account-related code is close together.

Simple rule:

If the project is small, layer-based organization is fine. If the project is growing, feature-based organization often becomes easier to maintain.

4. Module Boundaries

A module boundary answers this question:

What belongs together, and what should stay separate?

Imagine a banking app with these areas:

```
accounts  
transfers
```

```
customers
notifications
```

Each area should have its own responsibility.

Bad design:

```
public class AccountService {
    public void transferMoney() { }
    public void sendEmail() { }
    public void generateCustomerReport() { }
}
```

This class is doing too much.

Better design:

```
public class AccountService {
    public void openAccount() { }
    public void closeAccount() { }
}
```

```
public class TransferService {
    public void transferMoney() { }
}
```

```
public class NotificationService {
    public void sendEmail() { }
}
```

Each module should have a clear purpose.

5. Dependency Separation

A dependency is something your code relies on.

Example:

```
public class OrderService {
    private PaymentService paymentService;
}
```

OrderService depends on PaymentService .

Good dependency direction matters.

Usually, high-level business logic should not depend too heavily on low-level details.

Bad example:

```
public class ReportService {  
    private FileWriter fileWriter;  
}
```

This makes `ReportService` directly tied to writing files.

Better:

```
public interface ReportExporter {  
    void export(String content);  
}
```

```
public class FileReportExporter implements ReportExporter {  
    public void export(String content) {  
        // write to file  
    }  
}
```

```
public class ReportService {  
    private ReportExporter exporter;  
  
    public ReportService(ReportExporter exporter) {  
        this.exporter = exporter;  
    }  
}
```

Now `ReportService` does not care whether the report goes to a file, database, API, or email. That separation makes the code easier to test and change.

6. Naming and Organization Patterns

Good project structure depends heavily on clear names.

Weak names:

```
Utils.java  
Manager.java  
Processor.java  
Helper.java  
Data.java
```

These names are often too vague.

Better names:

```
AccountValidator.java  
TransferCalculator.java  
CustomerRepository.java
```

```
PaymentRequest.java
InvoiceFormatter.java
```

A class name should tell you what responsibility the class owns.

A package name should tell you what business or technical area the code belongs to.

7. Collaboration Standards

Project structure is also about teamwork.

On a team, everyone should agree on:

- where new classes go
- how packages are named
- where tests belong
- how configuration files are organized
- how large a class can become before it should be split
- how dependencies should flow between packages

Without shared structure, a codebase becomes confusing quickly.

A good team rule might be:

Every service class must have a matching test class under `src/test/java`.

Example:

```
src/main/java/com/example/banking/accounts/AccountService.java
src/test/java/com/example/banking/accounts/AccountServiceTest.java
```

That convention makes testing easier to find and maintain.

Mini Example

Suppose we are building a simple library system.

A reasonable structure might be:

```
src/main/java/com/example/library/
books/
  Book.java
  BookService.java
  BookRepository.java
members/
  Member.java
  MemberService.java
  MemberRepository.java
loans/
  Loan.java
  LoanService.java
notifications/
  NotificationService.java
```

This tells us:

- book logic belongs in `books`
- member logic belongs in `members`
- borrowing and returning logic belongs in `loans`
- email or SMS logic belongs in `notifications`

The project becomes easier to understand before you even open a file.

Practice Exercises

Exercise 1: Create a Standard Java Project Layout

Create this folder structure manually:

```
online-shop/  
  src/  
    main/  
      java/  
      resources/  
    test/  
      java/  
      resources/
```

Then add a package root:

```
src/main/java/com/example/shop
```

Goal: understand where production code, test code, and resources belong.

Exercise 2: Organize by Feature

Design packages for an online shopping app:

```
com.example.shop.products  
com.example.shop.customers  
com.example.shop.orders  
com.example.shop.payments  
com.example.shop.shipping  
com.example.shop.notifications
```

Inside each package, add two or three class names you would expect.

Example:

```
orders/  
  Order.java  
  OrderItem.java  
  OrderService.java
```

Goal: learn feature-based organization.

Exercise 3: Organize by Layer

Take the same shopping app and reorganize it by technical layer:

```
com.example.shop.model  
com.example.shop.service  
com.example.shop.repository  
com.example.shop.validator
```

Then compare it with feature-based organization.

Goal: understand the tradeoff between layer-based and feature-based structure.

Exercise 4: Split a Large Class

Start with a badly designed class:

```
public class ShoppingApp {  
    public void addProduct() {}  
    public void placeOrder() {}  
    public void processPayment() {}  
    public void sendEmail() {}  
    public void shipOrder() {}  
}
```

Split it into focused classes:

```
ProductService  
OrderService  
PaymentService  
NotificationService  
ShippingService
```

Goal: practice separation of responsibilities.

Exercise 5: Define Package Boundaries

For a banking app, decide which package owns each responsibility:

- open account
- close account
- transfer money
- send email confirmation
- validate customer identity
- calculate overdraft fee

Possible packages:

```
accounts  
transfers  
customers
```



```
notifications
fees
```

Goal: learn what belongs together and what should stay separate.

Exercise 6: Create Interface-Based Separation

Create this interface:

```
public interface NotificationSender {
    void send(String message);
}
```

Then create two implementations:

```
public class EmailNotificationSender implements NotificationSender {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}
```

```
public class SmsNotificationSender implements NotificationSender {
    public void send(String message) {
        System.out.println("SMS: " + message);
    }
}
```

Then use the interface inside another class:

```
public class OrderService {
    private NotificationSender notificationSender;

    public OrderService(NotificationSender notificationSender) {
        this.notificationSender = notificationSender;
    }

    public void placeOrder() {
        notificationSender.send("Order placed successfully");
    }
}
```

Goal: practice dependency separation.

Exercise 7: Match Test Structure to Main Structure

If you have:

```
src/main/java/com/example/shop/orders/OrderService.java
```

Create:

```
src/test/java/com/example/shop/orders/OrderServiceTest.java
```

Goal: learn professional test organization.

Exercise 8: Naming Cleanup

Rename vague classes into meaningful ones.

Bad names:

```
Manager.java  
Helper.java  
Data.java  
Processor.java  
Utils.java
```

Better names:

```
OrderService.java  
PaymentValidator.java  
CustomerProfile.java  
InvoiceCalculator.java  
EmailFormatter.java
```

Goal: improve readability through naming.

Exercise 9: Draw a Dependency Map

For this app:

```
orders  
payments  
inventory  
notifications
```

Decide which package can depend on which.

Example:

```
orders -> payments  
orders -> inventory  
orders -> notifications  
payments -> notifications
```

Then ask: should `notifications` depend on `orders` ? Usually no.

Goal: understand dependency direction.

Exercise 10: Mini Project

Build a small console-based Library Management System with this structure:

```
com.example.library.books  
com.example.library.members  
com.example.library.loans  
com.example.library.notifications
```

Minimum classes:

```
Book  
BookService  
Member  
MemberService  
Loan  
LoanService  
NotificationService  
Main
```

Required behavior:

- add a book
- register a member
- borrow a book
- return a book
- print a simple message when a book is borrowed

This is the best full practice exercise for Module 8 because it forces you to organize classes before the project becomes messy.

Lab: Structure a Java Library App

Build a small console-based Java project called Library Manager.

The goal is not advanced logic. The goal is clean project structure, packages, class boundaries, and dependency separation.

Lab Goal

Create a Java project organized like this:

```
library-manager/  
  src/  
    main/  
      java/  
        com/example/library/  
          Main.java  
          books/  
          members/  
          loans/
```

```
        notifications/
resources/
test/
    java/
        com/example/library/
resources/
```

Part 1: Create Packages

Create these packages:

```
com.example.library.books
com.example.library.members
com.example.library.loans
com.example.library.notifications
```

Each package should own one area of the app.

Part 2: Create Domain Classes

In `books`, create:

```
package com.example.library.books;

public class Book {
    private String title;
    private String author;
    private boolean borrowed;

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
        this.borrowed = false;
    }

    public String getTitle() {
        return title;
    }

    public boolean isBorrowed() {
        return borrowed;
    }

    public void borrow() {
        borrowed = true;
    }

    public void returnBook() {
        borrowed = false;
    }
}
```

```
}  
}
```

In `members` , create:

```
package com.example.library.members;  
  
public class Member {  
    private String name;  
  
    public Member(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Part 3: Create Service Classes

In `notifications` , create:

```
package com.example.library.notifications;  
  
public class NotificationService {  
    public void send(String message) {  
        System.out.println("Notification: " + message);  
    }  
}
```

In `loans` , create:

```
package com.example.library.loans;  
  
import com.example.library.books.Book;  
import com.example.library.members.Member;  
import com.example.library.notifications.NotificationService;  
  
public class LoanService {  
    private NotificationService notificationService;  
  
    public LoanService(NotificationService notificationService) {  
        this.notificationService = notificationService;  
    }  
  
    public void borrowBook(Book book, Member member) {  
        if (book.isBorrowed()) {  
            System.out.println("Book is already borrowed.");  
            return;  
        }  
    }  
}
```

```

    }

    book.borrow();
    notificationService.send(member.getName() + " borrowed " + book.getTitle());
}

public void returnBook(Book book, Member member) {
    book.returnBook();
    notificationService.send(member.getName() + " returned " + book.getTitle());
}
}

```

Part 4: Create Main Class

In `com.example.library`, create:

```

package com.example.library;

import com.example.library.books.Book;
import com.example.library.members.Member;
import com.example.library.loans.LoanService;
import com.example.library.notifications.NotificationService;

public class Main {
    public static void main(String[] args) {
        Book book = new Book("Clean Code", "Robert C. Martin");
        Member member = new Member("Ava");

        NotificationService notificationService = new NotificationService();
        LoanService loanService = new LoanService(notificationService);

        loanService.borrowBook(book, member);
        loanService.returnBook(book, member);
    }
}

```

Expected output:

```

Notification: Ava borrowed Clean Code
Notification: Ava returned Clean Code

```

Part 5: Refactor Challenge

Improve the structure by replacing `NotificationService` with an interface.

Create:

```

package com.example.library.notifications;

public interface NotificationSender {

```

```
    void send(String message);  
}
```

Then create:

```
package com.example.library.notifications;  
  
public class ConsoleNotificationSender implements NotificationSender {  
    public void send(String message) {  
        System.out.println("Notification: " + message);  
    }  
}
```

Change `LoanService` so it depends on `NotificationSender` , not the concrete class.

Success Checklist

You completed the lab if:

- production code is under `src/main/java`
- packages match folder structure
- each class has one clear responsibility
- `LoanService` does not contain book, member, or notification class definitions
- `Main` only wires objects together and runs the flow
- notification behavior is separated from loan logic

Extra Practice

Add a `BookService` with methods:

```
addBook(Book book)  
findBookByTitle(String title)  
printAllBooks()
```

This will help you practice deciding which logic belongs in `books` and which logic belongs in `loans` .

Key Takeaway

Module 8 is about learning to think like a professional Java developer:

Code should not just work. It should be organized so another developer can understand it, test it, and safely change it later.